Table of Contents

SupportBoard_sync.6.4

SupportBoard_sync.6.5

SupportBoard_sync.6.6

SupportBoard_theory.2.1

SupportBoard_theory.2.2

SupportBoard_theory.2.3

SupportBoard_utilbits.3.01

SupportBoard_utilbits.3.02

SupportBoard_utilbits.3.03

SupportBoard_utilbits.3.04

SupportBoard_utilbits.3.05

SupportBoard_utilbits.3.06

SupportBoard_utilbits.3.07

SupportBoard_utilbits.3.08

SupportBoard_utilbits.3.09

SupportBoard_utilbits.3.10

EEPROMDefTopic

< >

DIB and Probe Card EEPROMs

DIB and Probe Card EEPROMs

EEPROMS allow you to store information about a DIB and on probe cards. Encoded within each EEPROM is the name of the company that developed the board, the board type number, the board serial number, and the date of the last design revision.

This section includes the following topics:

- EEPROM Theory of Operation

- Programming EEPROMs

For DIB and Probe Card VBT language reference, see:

- TheHdw.DIB.EEPROM

- TheHdw.DIB.ProbeCardEEPROM

For more information on DIB EEPROMs on the Support Board, see DIB EEPROMs.

Safety

See About Operator and Safety Information for general safety requirements when working with an UltraFLEX test system.

SupportBoardDefTopic

< >

About the Support Boards

About the Support Boards

The information in this section relates to the usage of the UltraFLEX Support Boards, which are located in test head slots 24 and 25. The content is divided into the theory, functionality, and programmability of Utility Data Bits, User Power, DIB EEPROM, and Sync Panel.

The Support Boards provide two functions for the UltraFLEX tester:

- Instrument support functions are capabilities required by test system instruments in the system that are not programmable. These capabilities are not described here.
- User functions are instrument capabilities that are needed for use with all instruments, or capabilities too small to require using a test head slot for. These capabilities are described here.

This section includes the following topics:

- Features
- Safety Information
- Simulated Configuration Files
- Support Board Software Licensing Requirements

Support Board information includes the following sections:

- Support Board Theory of Operation
- Utility Data Bits
- DIB Power
- DIB and Probe Card EEPROMs
- Sync Panel

Related Topics

- UltraFLEX System Specifications
- VBT syntax:
  - TheHdw.DIB.EEPROM
  - TheHdw.DIB.PIBEEPROM
  - TheHdw.DIB.Power
  - TheHdw.DIB.ProbeCardEEPROM
  - TheHdw.DIB.SupportBoardClock
  - TheHdw.SyncPanel
  - TheHdw.Utility
- Support Board DIB Design Guidelines

SupportBoard_about.1.2

< >

About the Support Boards : Features

Features

The two UltraFLEX support boards are located in test head slots 24 and 25. Each board has an attached power board and a mechanical stiffener frame. The support board's main function is to support other hardware in the test system.

The following features are programmable:

- Utility data bits
- User power
- DIB and probe card EEPROMs

< >

SupportBoard_about.1.3

< >

About the Support Boards : Safety Information

Safety Information

Warning:

If you connect the user power supplies in series for increased voltage, high voltages or currents or both may be present in the device interface. Contacting this area of the equipment can cause personal injury, damage to the device under test (DUT), or damage to external equipment.

Caution:

Do not connect the 5 V user power supplies in parallel. Connecting them in parallel will damage the 5 V user power supplies. Use separate buses if your circuit needs more current.

When using the support boards, always follow general UltraFLEX test system safety guidelines. See Test System Safety.

< >

SupportBoard_about.1.4

<	>

About the Support Boards : Simulated Configuration Files

Simulated Configuration Files

You can use the Support Board and its functions offline by forcing IG-XL to map the Support Board. Do this by creating a SimulatedConfig.txt file and placing the file in any of the following locations:

- Folder containing the IG-XL workbook
- \tester folder: C:\Program Files (x86)Teradyne\IG-XL\<version>\tester
- \bin folder: C:\Program Files (x86)\Teradyne\IG-XL\<version>\bin

The following SimulatedConfig.txt file maps the primary Support Board board in slot 24 and the secondary Support Board in slot 25:

```
# UltraFLEX IG-XL hardware configuration
# This file contains an example hardware configuration, and
# is used as the default offline.

<Version>
1
</Version>

<TEST_HEAD TH 1>
#slot[.subslot] Type          idprom (type serial rev company)
-1.0       dib              239-001-00 0000000 0000-A 0000
1.0        HSD-M              979-219-00 0000000 0000-A 0000
           HSD-M            956-361-00 0000000 0000-A 0000
```

6.0 DC-30 805-002-00 0000000 0000-A 0000
DC-30 949-901-00 0000000 0000-A 0000
12.0 BBAC-15 979-214-00 0000000 0000-A 0000
BBAC-15 956-375-00 0000000 0000-A 0000
24.0 SupportBoard 974-220-12 0000000 0000-A 0000
SupportBoard 956-354-00 0000000 0000-A 0000
25.0 SupportBoard 979-220-01 0000000 0000-A 0000
SupportBoard 956-354-01 0000000 0000-A 0000
</TEST_HEAD TH 1>

To use the Support Board offline, the SimulatedConfig.txt file must include every board needed in the IG-XL workbook. The SimulatedConfig.txt file represents the state of the offline tester. To use the Support Board online, you need not edit any files. IG-XL automatically maps all boards in the tester.

For more information, see Tester Configuration.

SupportBoard_about.1.5

< >

About the Support Boards : Support Board Software Licensing Requirements

Support Board Software Licensing Requirements

The Support Board and its features do not require a software license.

For more information, see IG-XL Licensing in IG-XL Administration help.

< >

SupportBoard_dibpwr.4.1

<	>

DIB Power

DIB Power

The UltraFLEX test system has seven DC power supplies at the test head, called DIB Power, for your use. These power supplies are sometimes called “user power.”

DIB power is supplied by the master support board in test head slot 24. The available voltages are:

- 3.3V supply
- Two 5V supplies
- 12V supply
- Two 15V supplies
- 48V supply

All the power supplies are floating. They can be made positive or negative by grounding either the negative or positive output, and they can be connected in series for increased voltage. They cannot be connected in parallel for more current.

The following topics are available:

- DIB Power Theory of Operation
- DIB Power Programming

- DIB Power Debug Display

- DIB Power Applications

For more information, see:

- UltraFLEX System Specifications

- Power (VBT Syntax Reference)

- User Power Supplies (DIB Design Guidelines)

Caution:

Do not connect any of the DIB power supplies in parallel. Connecting them in parallel may damage the power supplies. Use separate buses if your circuit needs more current.

Warning:

If you connect the DIB power supplies in series for increased voltage, high voltages or currents or both may be present in the device interface. Contacting this area of the equipment can cause personal injury, damage to the device under test (DUT), or damage to external equipment.

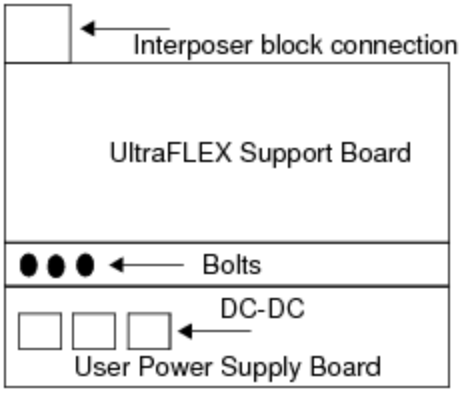
SupportBoard_dibpwr.4.2

< >

DIB Power : DIB Power Theory of Operation

DIB Power Theory of Operation

The DIB User Power Supply board (also called User Power Supply) is bolted to the UltraFLEX Support Board. The DIB Power Supply outputs pass through the Support Board from the bolt connection, then pass through the interposer block connection for use.



DIB or User Power Supply Connection

In most test cases, circuits must be placed on the DIB. These circuits usually add some signal processing capability that cannot be provided from the tester. Common examples are relays near the DUT to send the DUT signal to two different instrument types, with no additional loading on the pin, and op amp circuits to buffer a low level signal such that other instrumentation can make a measurement. These relays and op amps must be powered. User power provides these voltages.

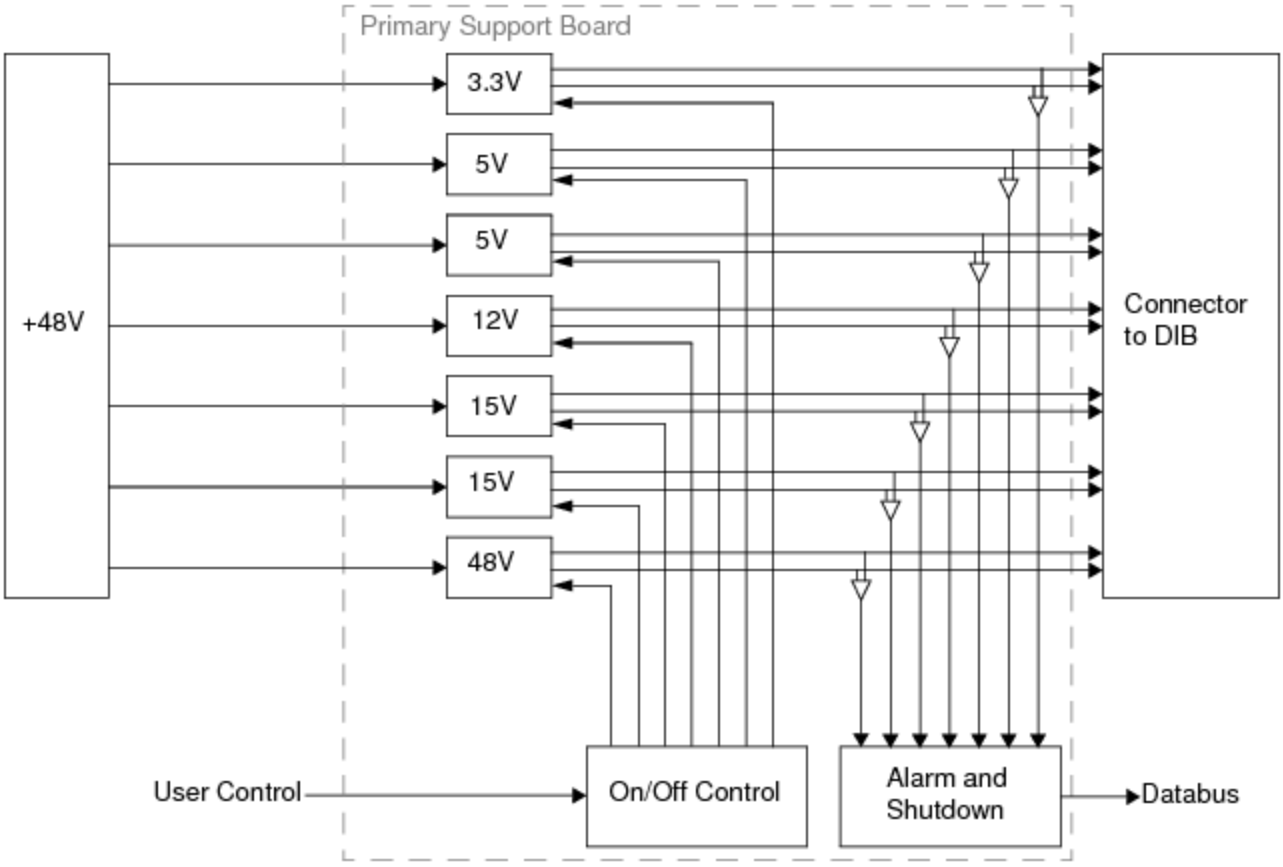
Specifications

The DIB power supplies are floating, enabling them to output either a negative or a positive voltage depending on the ground. If the high side is grounded the power supplies output a negative voltage, and if the low side is grounded the output is positive. Supplies are stackable for increased voltage. The primary Support Board in test head slot 24 provides the following user supplies:

- 3.3V (generally used for powering digital circuits)

- 5V A (generally used for powering digital circuits)
- 5V B (generally used for powering digital circuits)
- 12V (generally used for powering relays)
- 15V A (generally used for powering analog circuits)
- 15V B (generally used for powering analog circuits)
- 48V (generally used for powering analog circuits)

The 15V supplies are filtered to provide an electrically quieter signal to the DIB.
For specifications, see UltraFLEX System Specifications.
Circuit Description
The Support Board has six DC-DC converters. See Specifications.
The high and low outputs of the converters are driven to the DIB. Since each DIB signal path can handle only 0.5A, multiple signal pins are used as appropriate for the supply rating.
Converters have a control pin for turning the output voltages on and off. These control pins drive from an FPGA to allow software to control their state. Each converter has a DC OK signal that is monitored and sets a latch if it de-asserts during test program execution.



User Power Block Diagram

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_dibpwr.4.3

< >

DIB Power : DIB Power Programming

DIB Power Programming

You program the DIB Power using VBT syntax.

For more information on DIB Power VBT syntax, see Power (TheHdw.DIB.Power) in the VBT Syntax Reference.

Turn Power On or Off

To turn all of the power supplies on or off, use TheHdw.DIB.PowerOn. You can get or set the state of all power supplies by using a Boolean where True means on and False means off.

To turn any of the seven individual DIB power supplies on or off, use these VBT statements:

TheHdw.DIB.Power.Item("3.3V").State = tlOn | tlOff

TheHdw.DIB.Power.Item("5V_1").State = tlOn | tlOff

TheHdw.DIB.Power.Item("5V_2").State = tlOn | tlOff

TheHdw.DIB.Power.Item("12V").State = tlOn | tlOff

TheHdw.DIB.Power.Item("15V_2").State = tlOn | tlOff

TheHdw.DIB.Power.Item("15V_2").State = tlOn | tlOff

TheHdw.DIB.Power.Item("48V").State = tlOn | tlOff

Note that when you execute these properties without a DIB present and docked to the test head, then IG-XL returns the following error message and alarm:

Internal Error: A support board's A_CM_ERR register was set to 0x000C, but no contributing local error register was found.

DIB: 0004: Voltage 48V is Alarming. It has gone above its set threshold.

To avoid this error and alarm, you must always make sure that the DIB is present and docked to the test head.

If you use the User Power lines to create logic levels, note that setting a line to tlOn turns on the relay and causes the Open Drain output to go low. This creates an output of 0V. To program logic True, set the line to tlOff. To program logic False, set the line to tlOn.

Read Back Voltage or State

For each of the DIB power supplies, you can read back the voltage and state:

TheHdw.DIB.Power.Item("3.3V").Reading

TheHdw.DIB.Power.Item("3.3V").State

Set Alarm

For each of the DIB power supplies you can set the alarm:

TheHdw.DIB.Power.Item("3.3V").Alarm(tlDIBPowerAlarmHi | tlDIBPowerAlarmLo)

< >

2015 Teradyne, Inc. All rights reserved.

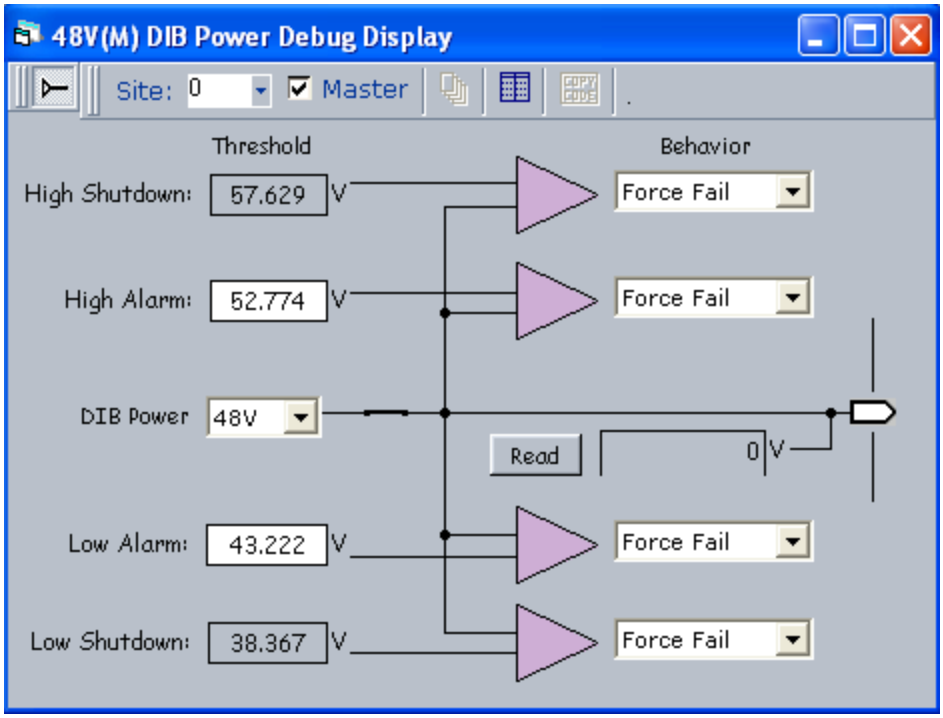
SupportBoard_dibpwr.4.4

< >

DIB Power : DIB Power Debug Display

DIB Power Debug Display

IG-XL includes a debug display for DIB Power. This display is an interactive, window-based tool which assists in debugging test programs that use DIB Power. Use the display to determine the voltage and state of DIB Power and to interactively set DIB Power parameters.



DIB Power Debug Display

To use the DIB Power Debug Display you must have a test program loaded and compiled. To launch the display, on the IG-XL Tools tab, in the Launch group, select Debug Displays>DIB Power.

You can also start the display from inside TDE.

The display has the following controls and properties:

Site	The site currently displayed by the debug display. Use the Site menu to select a different site.
Summary	On the TDE tool bar, the summary button is the one that appears to be a grid. Click this button to bring up the summary view. This is a read-only view with the data. Clicking a pin name causes the display to go back to the schematic view with that pin selected.
DIB Power	Select the power supply to program. The options are: • 48V • 15_2V: The second of two 15 V power supplies • 15_1V: The first of two 15 V power supplies • 12V • 5_2V: The second of two 5 V power supplies • 5_1V: The first of two 5V power supplies • 3.3V
High Alarm	The positive voltage at which an alarm is returned.
Low Alarm	The negative voltage at which an alarm is returned.
High Shutdown	The positive voltage at which the DIB Power is shut down. This is read-only and cannot be changed.
Low Shutdown	The negative voltage at which the DIB Power is shut down. This is read-only and cannot be changed.
Behavior	Can be: • Force Fail • Force Bin

	• Off • Default • Continue • Raise Error
Force Fail	Causes a failure if an alarm or shutdown occurs
Force Bin	Cause the device to be binned if an alarm or shutdown occurs.
Off	Turns off DIB Power if an alarm or shutdown occurs.
Default	Selecting Default applies the Force Fail selection. It is included to allow you to quickly return to the default settings.
Continue	Test program continues if an alarm or shutdown occurs.
Raise Error	Test program returns an error if an alarm or shutdown occurs.
Read	Click this button to display the voltage in the box next to the button.

For more information on using debug displays, see the following:

- Using Debug Displays in TDE
- Producing Code from Debug Displays

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_dibpwr.4.5

<	>

DIB Power : DIB Power Applications

DIB Power Applications

This section describes typical applications for DIB Power.

VBT Programming Example

```
' Before turning on power, you must always make sure that the DIB is present
' and docked to the test head to avoid errors and alarms.

' Turn on all DIB power supplies
TheHdw.DIB.PowerOn=True
' Turn off all DIB power supplies
TheHdw.DIB.PowerOn=False
' Note that the DIB power names are fixed as seen here
' Turn on one of the DIB power supplies
TheHdw.DIB.Power ("5V_1").State=tlOn
TheHdw.DIB.Power ("5V_2").State=tlOn
TheHdw.DIB.Power ("15V_1").State=tlOn
TheHdw.DIB.Power ("15V_2").State=tlOn
TheHdw.DIB.Power ("12V").State=tlOn
TheHdw.DIB.Power ("3.3V").State=tlOn
TheHdw.DIB.Power ("48V").State=tlOn
' Turn off one of the DIB power supplies
TheHdw.DIB.Power ("5V_1").State=tlOff
TheHdw.DIB.Power ("5V_2").State=tlOff
TheHdw.DIB.Power ("15V_1").State=tlOff
TheHdw.DIB.Power ("15V_2").State=tlOff
TheHdw.DIB.Power ("12V").State=tlOff
```

```
TheHdw.DIB.Power ("3.3V").State=tlOff
TheHdw.DIB.Power ("48V").State=tlOff
' Read the actual supplied voltage of a particular DIB power supply
Dim Actual_5V As Double
Actual_5V = TheHdw.DIB.Power ("5V_1").Reading
' Set the alarm upper voltage limit for a particular DIB power supply
TheHdw.DIB.Power("5V_1").Alarm(tlDIBPowerAlarmHi).Threshold = 5.51
' Set the alarm lower voltage limit for a particular DIB power supply
TheHdw.DIB.Power("5V_1").Alarm(tlDIBPowerAlarmLo).Threshold = 4.45
' Set the alarm behavior
TheHdw.DIB.Power ("5V_1").Alarm.Behavior=tlAlarmForceFail
' Get the state for a particular DIB power supply
' True indicates that supply is ON, False indicates supply is OFF.
Dim Power_Supply_State As Boolean
Power_Supply_State = TheHdw.DIB.Power("5V_1").Power.State
```

< >

SupportBoard_eeprom.5.02

< >

DIB and Probe Card EEPROMs : EEPROM Theory of Operation

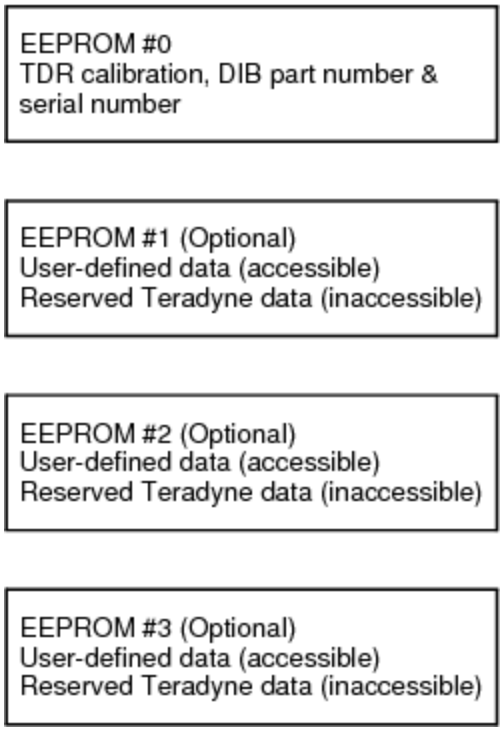
EEPROM Theory of Operation

Up to four EEPROMs, #0-3, can be used on DIBs and probe cards in any combination. For example, you could have three EEPROMs on the DIB and one EEPROM on the probe card or two EEPROMs on the DIB and two EEPROMs on the probe card, and so on.

The first EEPROM (#0) is used to store header information and calibration. Information stored on this EEPROM pertains only to the test system and includes TDR calibration, DIB board part number, and serial number storage. You cannot write to it except for the serial number and part number.

To store user data (device or test information), you need more than one EEPROM on the DIB. You can use EEPROMs #1-3 to store this type of data. Approximately 20% of memory space on these EEPROMs is preallocated for your use. As you use memory space, IG-XL automatically allocates more space until all available memory space on the EEPROM is used.

The following figure shows the EEPROMs on a DIB.



EEPROMs in a Test System

☑ **Note:**

Design DIBs and probe cards with the number of EEPROMs that may be required in the future. Adding more EEPROMs later requires initialization of the EEPROMs to write the necessary header information on all EEPROMs, which overwrites any existing data. To avoid loss of data, install additional EEPROMs when the DIBs or probe cards are first used.

IG-XL uses all available space on all EEPROMs on a DIB as one contiguous memory space. Similarly, IG-XL uses all available space on the EEPROMs on a probe card as another, but separate, contiguous memory space.

This section includes the following topics:

- EEPROMs on a DIB
- EEPROM Data Format

Read and write operations are programmable using VBT syntax. For more information, see:

- TheHdw.DIB.EEPROM
- TheHdw.DIB.ProbeCardEEPROM

SupportBoard_eeprom.5.03

< >

DIB and Probe Card EEPROMs : EEPROM Theory of Operation : EEPROMs on a DIB

EEPROMs on a DIB

A DIB can contain up to four EEPROMs, although you can only write user data to EEPROMs #1-3. EEPROMs can be programmed to hold DIB information, such as DIB part number, device type, manufacture date, calibration information, and so on.

For information on the circuitry and DIB EEPROM layout requirements, see DIB EEPROMs.

☑ **Note:**

At least one EEPROM circuit must be installed on a DIB.

< >

SupportBoard_eeprom.5.04

< >

DIB and Probe Card EEPROMs : EEPROM Theory of Operation : EEPROM Data Format

EEPROM Data Format

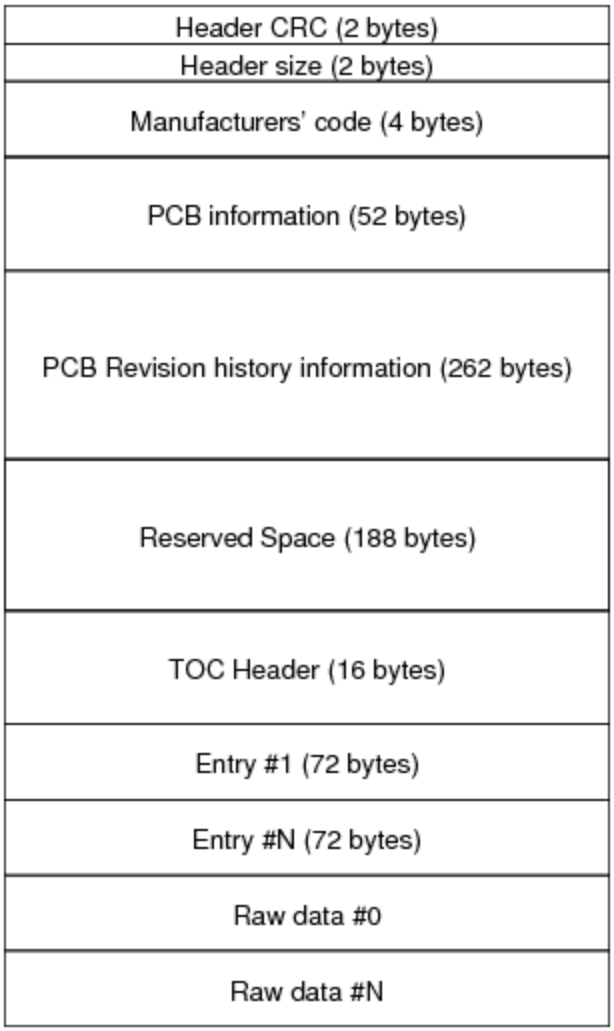
The first 512 bytes of each EEPROM are allocated for the board ID information such as:

- Serial number
- Part number
- Manufacturing date
- Latest revision date
- Revision level

Starting at address 512, an EEPROM includes a table of contents containing data information. The TOC consists of TOC Header, TOC entries, and raw data blocks, each of which is described and pointed to in the corresponding TOC entry.

You can use the memory space in the owner block of each EEPROM (#1-3) to record any information you wish. All other locations in the EEPROMs are reserved for Teradyne's use and are described here for reference only.

Identification information stored in an EEPROM is organized as shown in the figure.



Organization of Data in DIB and Probe Card EEPROMs

Header CRC

The first location in an EEPROM contains a 16 bit cyclical redundancy check (CRC) for the header.

Header Size

This location contains the size, in bytes, of the header. The size of the header is used to read the raw header data whose CRC is compared with the stored CRC. A CRC mismatch causes an error to be returned and makes the EEPROM unavailable. The size is not controlled by the CRC.

Manufacturers' Code

This block is for use by Teradyne. If you create a DIB, this block is set to a default value. You can store information only in the owner block.

PCB Information

This block is used to information on the PCB such as the following:

- PCB block CRC
- PCB block size

- PCB serial number
- PCB Teradyne part number
- PCB revision level
- PCB latest revision date
- PCB manufacturing date

PCB Revision History Information

This block contains information on revisions to the PCB such as the following:

- PCB revision date
- PCB revision history block CRC
- PCB revision history block size
- PCB revision entry #0 - #62

Reserved Space

This memory is reserved and cannot be written to.

Table of Contents Header

The memory is divided into blocks with each client owning one or more blocks. The blocks are managed by this TOC. This block contains the following information:

- Number of bytes in NVRAM
- Number of bytes in header
- Number of entries in TOC
- Memory aligning pad

- CRC for this header

Records

You can use these blocks to store any information that you wish to maintain. For example, you can enter the DIB or probe card ID or part number in these blocks and in your channel map. IG-XL then reads the part number stored in this block and compares it to the value in the channel map. You must use the string ID, PartNumber, to allow IG-XL to directly access the information.

You can put in any number of records or as many as will fit into the block, leaving empty space if desired or if necessary. The number of records in the block is stored in one byte at the beginning of the block. This limits the number of records in one block to 255. If you run out of space in one block and if free space is available, IG-XL assigns another block for you. You can continue to use blocks until all space on the EEPROM is used. IG-XL then displays a notice that all memory has been used. For more memory space, you can add more EEPROMs, up to a total of four. The additional EEPROMs must be initialized, however, which overwrites all data in all EEPROMs. To avoid data loss, add the EEPROMs when the DIB is manufactured.

EEPROMs store data as a string ID and a data field.

Data types supported are:

- Integer (2 bytes)
- Long (4 bytes)
- Boolean (1 byte)
- Single (4 bytes)
- Double (8 bytes)
- Date (8 bytes)
- Byte
- String (size of string + 1 byte to store the length of the string)

SupportBoard_eeprom.5.05

<	>

DIB and Probe Card EEPROMs : Programming EEPROMs

Programming EEPROMs

You can interactively read or change the contents of DIB and probe card EEPROMs (EEPROMs 1-3) using VBT statements. For each VBT statement described in this section, you can write VBT functions to perform the task, or you can use the VBT statements in the immediate window.

☑ **Note:**

Topics in this section do not apply to EEPROM 0.

The following topics are available:

- Initializing DIB EEPROMs
- Specifying EEPROMs
- Saving the Contents of EEPROMs to a File
- Writing the Contents of a Backup File to an EEPROM
- Checking If an EEPROM Is Programmed
- Adding New Information to an EEPROM
- Writing to an EEPROM
- Retrieving a List of Records

- Retrieving an Individual Record from an EEPROM
- Deleting a Record from an EEPROM
- DIB EEPROM Programming Examples

For more information on EEPROM VBT programming, see:

- TheHdw.DIB.EEPROM
- TheHdw.DIB.ProbeCardEEPROM

< >

SupportBoard_eeprom.5.06

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Initializing DIB EEPROMs

Initializing DIB EEPROMs

Initialize a DIB EEPROM with this statement:

TheHdw.DIB.EEPROM.Program

This VBT statement automatically finds and initializes DIB EEPROMs with the default structure. With this statement, you need not specify an EEPROM or EEPROMs. After finding them, the statement initializes all EEPROMs (excluding EEPROM 0) and treats all the EEPROMs as one continuous address space. This statement does nothing if the EEPROMs already have the required default structure.

This statement is for test systems with DIB EEPROMs only. Do not use this statement on test systems with both DIB EEPROMs and probe card EEPROMs. If you do, all DIB EEPROMs and probe card EEPROMs are initialized to DIB EEPROMs. For test systems with both DIB EEPROMs and probe card EEPROMs, see Specifying EEPROMs.

EEPROMs must be in contiguous address space. This statement initializes EEPROMs from the lowest address space, 0, up to the maximum. Any EEPROMs preceded or separated by an unused location are not initialized.

DIB and probe card EEPROMs must have header and other information for IG-XL to recognize and read them. Teradyne writes this information onto all EEPROMs supplied by Teradyne. For EEPROMs from other sources you must write the header and other information to the EEPROM using these VBT initialization statements.

For more information, see TheHdw.DIB.EEPROM.Program in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.07

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Specifying EEPROMs

Specifying EEPROMs

For DIB EEPROMs and probe card EEPROMs:

```
Dim config (2) As Long
' Program DIB EEPROM #1 and #2 to be 32Kb
config(0) = 32768
config(1) = 32768
config(2) = 0
TheHdw.DIB.EEPROM.program (config)
' Program PIB for one 32K EEPROM
config(0) = 0
config(1) = 0
config(2) = 32768
TheHdw.DIB.PIBEEPROM.program (config)
```

Use these VBT statements to initialize EEPROMs on test systems with both DIB and probe card EEPROMs.

You must use these statements if your test system has a probe card EEPROM at other addresses, for example, bus address 2. Without both these statements, automatic detection groups the EEPROMs at bus addresses 0, 1, and 2 as one continuous address space even though you want the EEPROM at bus addresses 2 to be the probe card EEPROM.

These VBT statements allow you to select specific EEPROMs. For example, the above statements select EEPROMs at bus address 0 and 1 as the DIB EEPROMs and 2 as the probe card EEPROM. If you specify only one or two EEPROMs, all other EEPROMs, if any, are ignored.

For more information, see TheHdw.DIB.EEPROM.Program in the VBT Syntax Reference.

SupportBoard_eeprom.5.08

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Saving the Contents of EEPROMs to a File

Saving the Contents of EEPROMs to a File

Once you have written data to an EEPROM, you can read it back out to a file using the ArchiveToFile VBT syntax.

For DIB EEPROMs:

```
TheHdw.DIB.EEPROM.ArchiveToFile "backup"
```

For probe card EEPROMs:

```
TheHdw.DIB.ProbeCardEEPROM.ArchiveToFile "backup"
```

These VBT statements write the contents of an EEPROM to a file with the specified name. Files are created in the active workbook folder or current working directory.

The following example code shows archiving data from a DIB EEPROM:

```
Dim eeprom As ExecDIBEEPROM
Set eeprom = TheHdw.DIB.eeprom
eeprom.Record.Item("DIB1") = "123456" ' Record entry. Not yet input to EEPROM.
eeprom.Record.WriteToHW ' Make sure to input all items you wish to input to
                        ' EEPROM before executing this method. All entrys are
                        ' written to EEPROM.
EEPROM.ArchiveToFile("DIB1") ' Saves to directory your program is in. Archives
                             ' user data from EEPROMs #1-3
```

The following figure shows the DIB1.bin file. With this file, you can determine what user data is stored on user EEPROMS (#1-3).

```
00000200h: 00 7E 01 00 10 00 00 00 02 00 00 00 00 00 63 B5 ; .~............cµ
00000210h: 55 73 65 72 20 44 49 42 20 4E 56 52 41 4D 20 54 ; User DIB NVRAM T
00000220h: 4F 43 20 44 61 74 61 00 00 00 00 00 00 00 00 00 ; OC Data.........
00000230h: 00 00 00 00 00 00 00 00 10 00 00 00 01 00 00 00 ; ................
00000240h: 01 00 00 00 01 00 00 00 06 00 00 00 04 00 00 00 ; ................
00000250h: A0 02 00 00 6E 13 C1 C9 44 49 42 20 4E 56 52 41 ;  ...n.ÁÉDIB NVRA
00000260h: 4D 20 55 73 65 72 20 44 61 74 61 00 00 00 00 00 ; M User Data.....
00000270h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000280h: 01 00 00 00 01 00 00 00 01 00 00 00 01 00 00 00 ; ................
00000290h: 09 00 00 00 20 7D 01 00 E0 02 00 00 A4 B3 A7 FE ; .... )..à...¤³§þ
000002a0h: 00 80 00 00 00 80 00 00 00 80 00 00 01 00 00 00 ; .€...€...€......
000002b0h: 39 4C 00 00 01 00 00 00 07 00 00 00 00 00 00 00 ; 9L..............
000002c0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
000002d0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
000002e0h: 01 39 4C 00 00 00 00 01 00 04 01 44 49 42 31 06 ; .9L........DIB1.
000002f0h: 31 32 33 34 35 36 00 00 00 00 00 00 00 00 00 00 ; 123456..........
00000300h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
00000310h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ; ................
```

Sample EEPROM File

For more information, see TheHdw.DIB.EEPROM.ArchiveToFile in the VBT Syntax Reference.

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_eeprom.5.09

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Writing the Contents of a Backup File to an EEPROM

Writing the Contents of a Backup File to an EEPROM
For DIB EEPROMs:
TheHdw.DIB.EEPROM.LoadFromFile "backup"
For probe card EEPROMs:
TheHdw.DIB.ProbeCardEEPROM.LoadFromFile "backup"
These VBT statements write the contents of EEPROM files to EEPROMs. They take the contents of files, named, backup<I2C address>.bin, and write them to the corresponding EEPROMs. For example:
backup_0.bin is written to EEPROM at bus address 0
backup_1.bin is written to EEPROM at bus address 1
backup_2.bin is written to EEPROM at bus address 2
and so on.
Any information on the EEPROM is overwritten. The files must be in the active workbook folder.
For more information, see TheHdw.DIB.EEPROM.LoadFromFile in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.10

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Checking If an EEPROM Is Programmed

Checking If an EEPROM Is Programmed

For DIB EEPROMs:

Debug.Print TheHdw.DIB.EEPROM.IsProgrammed

For probe card EEPROMs:

Debug.Print TheHdw.DIB.ProbeCardEEPROM.IsProgrammed

These VBT statements determine if the EEPROM is programmed. They return TRUE if the EEPROM is programmed and FALSE if not.

For more information, see TheHdw.DIB.EEPROM.IsProgrammed in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.11

<	>

DIB and Probe Card EEPROMs : Programming EEPROMs : Adding New Information to an EEPROM

Adding New Information to an EEPROM

For DIB EEPROMs:

TheHdw.DIB.EEPROM.Record.Item("hello") = "world"

TheHdw.DIB.EEPROM.Record("long") = 123

TheHdw.DIB.EEPROM.Record("bool") = True

For probe card EEPROMs:

TheHdw.DIB.ProbeCardEEPROM.Record.Item("hello") = "world"

TheHdw.DIB.ProbeCardEEPROM.Record("long") = 123

TheHdw.DIB.ProbeCardEEPROM.Record("bool") = True

These VBT statements add new information to an EEPROM. The new information is held in a buffer. You must write the information to the EEPROM (see Writing to an EEPROM).

The first statement adds a record, named hello, that contains the string world. The second statement adds another record, 123, that is a long data type. The third statement adds a third record to the EEPROM, True, that is a Boolean.

Data types supported are:

- Integer (2 bytes)
- Long (4 bytes)
- Boolean (1 byte)
- Single (4 bytes)

- Double (8 bytes)
- Date (8 bytes)
- Byte
- String (size of string + 1 byte to store the length of the string)

For more information, see TheHdw.DIB.EEPROM.Record in the VBT Syntax Reference.

SupportBoard_eeprom.5.12

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Writing to an EEPROM

Writing to an EEPROM
For DIB EEPROMs:
TheHdw.DIB.EEPROM.Record.WriteToHW
For probe card EEPROMs:
TheHdw.DIB.ProbeCardEEPROM.Record.WriteToHW
These VBT statements allow you to add or update many records before saving to an EEPROM. This saves time and increases the life of the EEPROM. If the EEPROM does not have enough space for the records, an error is returned and some blocks must be deleted if you want to write the records, or you can add more EEPROMs up to a maximum of four on the DIB and probe card. Additional EEPROMs must be initialized which overwrites all data in all EEPROMs on the DIB. To avoid data loss, add DIB EEPROMs when the DIB is manufactured and first used.
Note that you can only write user data to EEPROMs #1-3. EEPROM #0 is reserved for TDR calibration data and for reading and writing serial and part numbers.
For more information, see TheHdw.DIB.EEPROM.Record.WriteToHW in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.13

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Retrieving a List of Records

Retrieving a List of Records

For DIB EEPROMs:

```
Dim rec() As IDIB_EEPROM_RecordObj
rec = TheHdw.DIB.EEPROM.Record.List
```

For probe card EEPROMs:

```
Dim rec() As IDIB_EEPROM_RecordObj
rec = TheHdw.DIB.ProbeCardEEPROM.Record.List
```

These VBT statements put the records in an EEPROM into rec as an array of IDs and values. The object in these VBT statements has two properties: ID and a value.

For more information, see TheHdw.DIB.EEPROM.Record.List in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.14

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Retrieving an Individual Record from an EEPROM

Retrieving an Individual Record from an EEPROM

For DIB EEPROMs:

```
Dim rec_val As Variant
rec_val = TheHdw.DIB.EEPROM.Record("hello")
```

For probe card EEPROMs:

```
Dim rec_val As Variant
rec_val = TheHdw.DIB.ProbeCardEEPROM.Record("hello")
```

To read an individual record from the EEPROM, use these VBT statements. The above example reads the record, hello and puts the value into rec_val.

For more information, see TheHdw.DIB.EEPROM.Record in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.15

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : Deleting a Record from an EEPROM

Deleting a Record from an EEPROM

For DIB EEPROMs:

```
Dim rec() As IDIB_EEPROM_RecordObj
TheHdw.DIB.EEPROM.Record.Delete "long", rec
TheHdw.DIB.EEPROM.Record.WriteToHW
```

For probe card EEPROMs:

```
Dim rec() As IDIB_EEPROM_RecordObj
TheHdw.DIB.ProbeCardEEPROM.Record.Delete "long", rec
TheHdw.DIB.ProbeCardEEPROM.Record.WriteToHW
```

These statements delete a record from the DIB EEPROM. In the above example, they delete the record with the string ID, long. The result is the array, rec.

For more information, see TheHdw.DIB.EEPROM.Record.Delete in the VBT Syntax Reference.

< >

SupportBoard_eeprom.5.16

< >

DIB and Probe Card EEPROMs : Programming EEPROMs : DIB EEPROM Programming Examples

DIB EEPROM Programming Examples

The following code examples are usable functions for interfacing with an EEPROM using VBT language:

- Adding New Information
- Listing Records
- Deleting a Record

For more information on EEPROM VBT programming, see:

- TheHdw.DIB.EEPROM
- TheHdw.DIB.ProbeCardEEPROM

Adding New Information

This example adds new information to a DIB EEPROM

```
Public Sub WriteEEPROM()
On Error GoTo errhand
Dim isProg As Boolean
Dim i As Long
   For i = 0 To 1
      isProg = TheHdw.DIB.EEPROM.IsProgrammed
```

```
        If (isProg = False) Then
        Debug.Print "PROBLEM"
        Else
        TheHdw.DIB.EEPROM.Record.Item("test") = "this is a test"
        TheHdw.DIB.EEPROM.Record.Item("dummy") = 0.3
        TheHdw.DIB.EEPROM.Record.WriteToHW
        End If
    Next i
    Exit Sub
errhand:
Debug.Print "ERROR"
Resume Next
End Sub
```

Listing Records

This example lists the records inside an EEPROM.

```
Public Sub PrintEEPROM()
On Error GoTo errhand
Dim rec() As IDIB_EEPROM_RecordObj
rec = TheHdw.DIB.EEPROM.Record.List
Dim i As Long
For i = LBound(rec) To UBound(rec)
    Debug.Print "ID: " & rec(i).ID & "   Value: " & rec(i).Value
Next i
Exit Sub
errhand:
Debug.Print "ERROR"
Resume Next
End Sub
```

Deleting a Record

This example deletes a record from a EEPROM

```
Public Sub DeleteEEPROM()
On Error GoTo errhand
Dim rec() As IDIB_EEPROM_RecordObj
Dim rec2() As IDIB_EEPROM_RecordObj
rec = TheHdw.DIB.EEPROM.Record.List
Dim i As Long
    For i = LBound(rec) To UBound(rec)
    TheHdw.DIB.EEPROM.Record.Delete rec(i).ID, rec2
```

```
    Next i
    TheHdw.DIB.EEPROM.Record.WriteToHW
    Debug.Print "Items Deleted"
Exit Sub
errhand:
Debug.Print "ERROR"
Resume Next
```

<	>

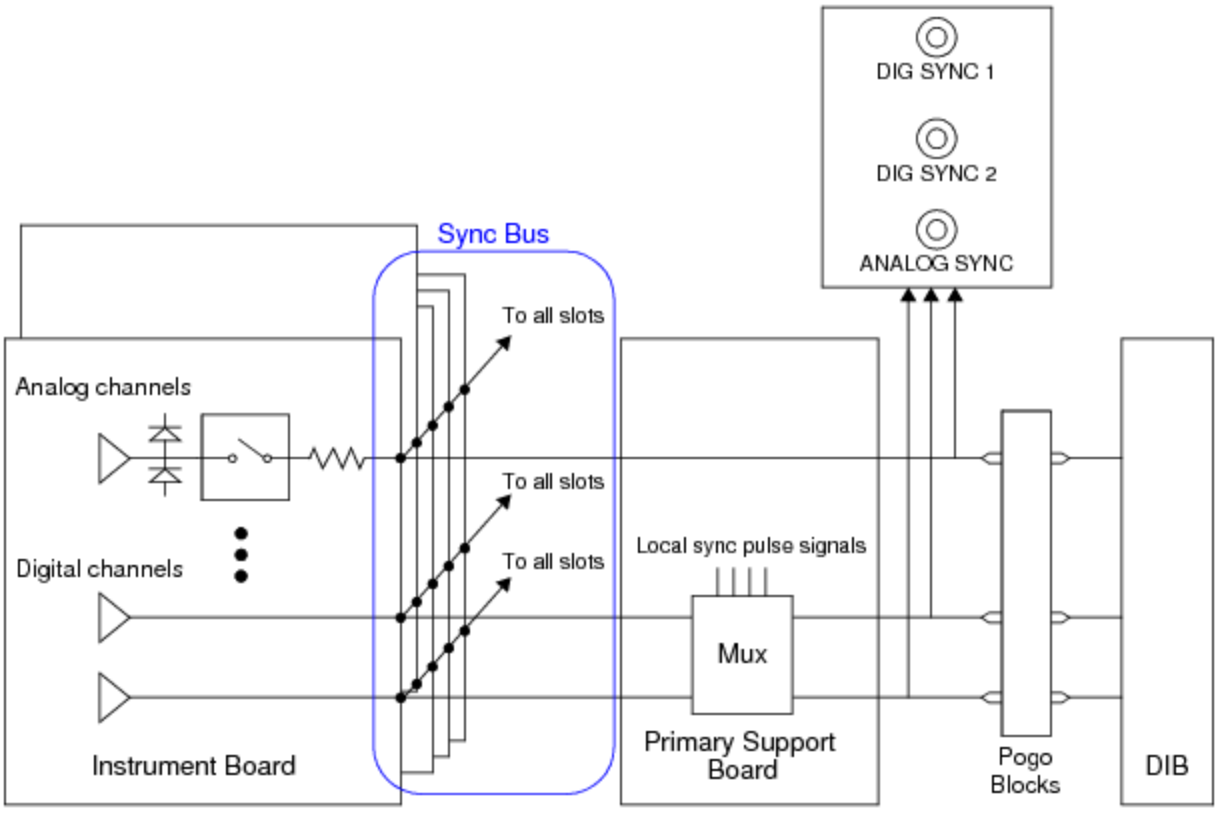
SupportBoard_sync.6.1

< >

Sync Panel

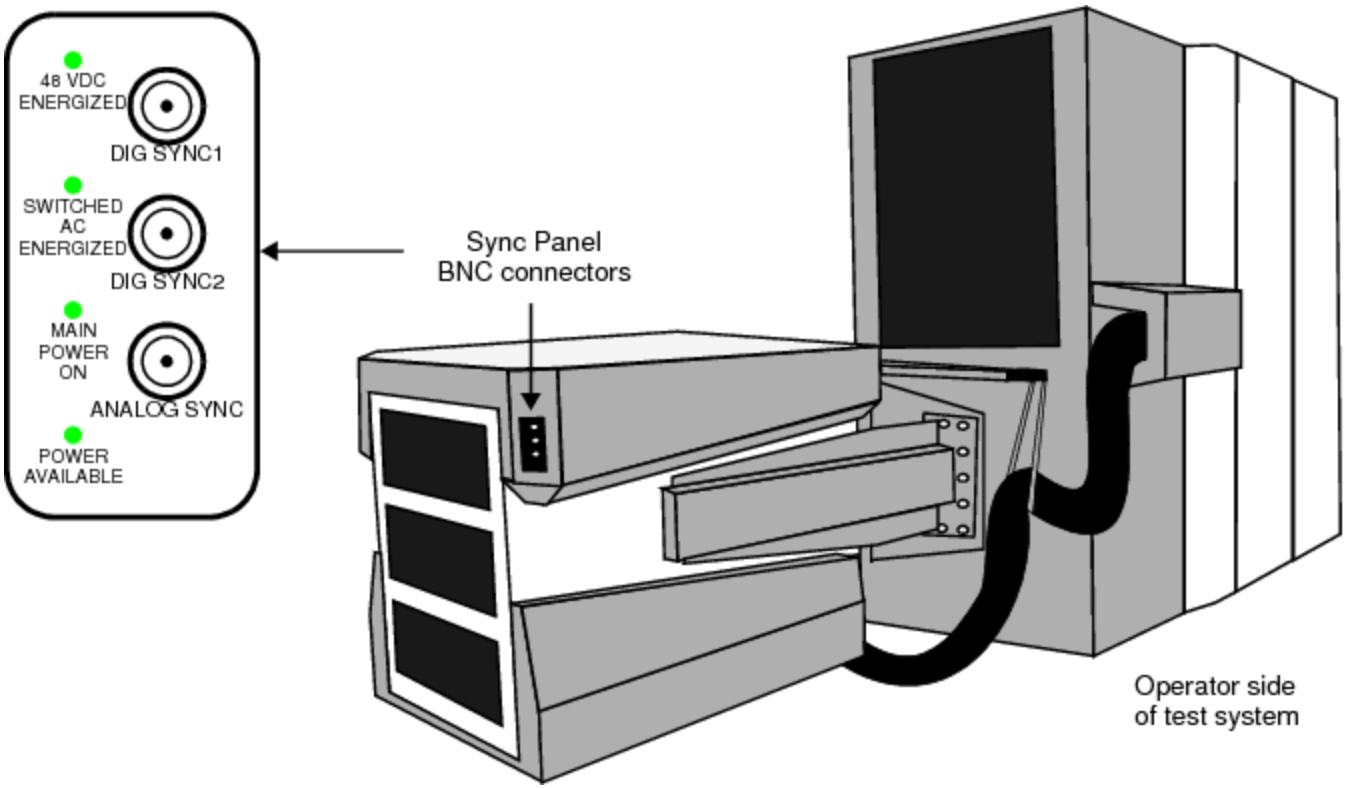
Sync Panel

The Sync Panel is a debug feature that allows you to connect an oscilloscope or other external instrument to BNC connectors on the test head and view internal test system signals while debugging a test program. The test system signals pass from the instrument that you selected, through the Support Board to the DIB area, and through the Support Board I/O connector to the DIB.



Typical Sync Panel Bus Connections

BNC connections on the test head Sync Panel include two digital connectors (DIG SYNC 1 and DIG SYNC 2) and one analog connector (ANALOG SYNC).



Sync Panel Connectors

The following table lists the available Sync Panel connections.

Sync Panel Connections Per Instrument and Signal

	Sync Panel or Sync Bus Line Availability		
Signal	DC30	DC75	UltraVI80
DCDiffMeter ADC Input	Analog	Analog	Analog
DCDiffMeter ADC Start	Dig1	Dig1	Dig1
DCTime Async Trigger 1	Dig1	Dig1	Dig1
DCTime Async Trigger 2	Dig1	Dig1	Dig1
DCTime Shutdown	NA	NA	Dig1
DCVI ADC Input	Analog	Analog	NA
DCVI ADC Start	Dig1	Dig1	Dig1
DCVI Alarm	NA	NA	Dig1

For more information, see:

- Starting the Sync Panel Debug Display

- Using the Sync Panel Debug Display

- SyncPanel (VBT Syntax Reference)

- Sync Panel, Uncovering Issues (2007 TUG paper) in eKnowledge

Digital Connectors

Some test system instruments can generate a pulse through one of the digital Sync Panel BNC connectors to trigger an external instrument. To monitor signals from a digital attribute of an analog channel, set the external instrument to trigger on a specific level or on a pulse, then tell IG-XL when to generate that pulse.

For DC30, DC75, and UltraVI80 channels, the pulse is a falling edge from 3.3V.

For UltraVI80 channels, digital sync signals are about 800ns (minimum). These signals can result in one long active low pulse if the ADC rate (for instance, the high-speed ADC with averaging less than 8) is greater than 1 megasample (after averaging).

For the Support Board sync (spot) pulse, the pulse is a rising edge to 3.3V.

Analog Connector

To monitor data from an analog channel, such as a DCVI channel, set the external instrument to trigger on an analog transition. This does not require a digital sync pulse.

The Sync Panel carries a maximum of 5V. For DC30 and DC75 channels, if a signal uses voltages greater than 5V, the Sync Panel scales them to the 5V range. For example, if your signal uses the 20V range and you program 15V, the Sync Panel shows 3.75V.

For UltraVI80 channels, in the 7V, 3.5V and 1.4V DCDiffMeter ranges, the Sync Panel scales them to 1/6, 1/3, and 5/6 of the DCDiffMeter reading respectively.

For DC30 and DC75 channels, Analog Sync Panel voltages are inverted. (UltraVI80 channels are not inverted.) For example, if you program 4V, the voltage at the Sync Panel BNC connector is -4V. The voltage at the device pin remains unchanged. Other analog connections represent voltages normally.

Typical Sync Panel Characteristics

- Sync Panel impedance: 50 ohms

- Capacitance: <500pF

- Maximum voltage

- Analog signals: 15V
- Digital signals: 0V to +3.3V

- Driver
  - Analog signals: AD711 or equivalent
  - Digital signals: CMOS, 16mA tristate
- Spot pulse width, digital signals: 10s

Path length: <12 feet

For more information, see UltraFLEX System Specifications.

SupportBoard_sync.6.2

<	>

Sync Panel : Programming the Sync Panel

Programming the Sync Panel

The Sync Panel begins relaying a signal as soon as you connect a line. While you can do this at any time, you may find it useful to monitor a signal only during a particular portion of your test program. To do this, you must trigger your external instrument to begin monitoring. You can configure the external instrument to trigger on a specific level or transition, or you can use a separate digital Sync Panel connection to deliver the trigger.

To connect a channel to the Sync Panel:

1.	Connect the channel to the Sync Panel line. Use the following VBT statement:

TheHdw.<inst>.Pins.ConnectToSyncPanel

where <inst> is the VBT interface for the instrument you want to connect. For example, to connect DCTime pin TimeOut1 to the Sync Panel, use the following command:

```
TheHdw.DCTime.Pins("TimeOut1").ConnectToSyncPanel _
                    tlFlexSyncPanelDigitalLine1, _
                    tlDCTimeSyncSignalAsyncTrigger1)
```

2.	Connect a trigger line to another Sync Panel output if you want to start the external instrument using a separate trigger pulse. See Using a Second Signal to Trigger an External Instrument for more information.
3.	Run your test program and monitor the output on the external instrument.
4.	Disconnect all channels from the Sync Panel. Use the Disconnect method:

TheHdw.SyncPanel.Disconnect(BusLine)

where BusLine is the same signal line you connected in the ConnectToSyncPanel statement. If you used two Sync Panel lines (trigger and data), disconnect both in separate Disconnect statements.

Connections from DC30 and DC75 instruments to the Sync Panel have the following restrictions:

- DC signals (DCVI ADC Start, DCDiffMeter ADC Start, and DCTime Async Trigger) cannot connect to DigitalLine2.

- You cannot connect multiple channels from one board to the Sync Panel. The analog and digital attributes must come from the same channel or from different boards.

Connections from UltraVI80 instruments to the Sync Panel have the following restriction:

- DC signals (DCVI ADC Start, DCVI Alarm, DCDiffMeter ADC Start, DCTime Async Trigger, and DCTime Shutdown) cannot connect to DigitalLine2.

You can perform all these actions, except generating the sync pulse, from the Sync Panel Debug Display. For more information, see Using the Sync Panel Debug Display.

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_sync.6.3

<	>

Sync Panel : Programming the Sync Panel : Using a Second Signal to Trigger an External Instrument

Using a Second Signal to Trigger an External Instrument

Because each Sync Panel output can connect to one channel at a time, using a separate trigger line requires two Sync Panel connections (one for the data signal and one for the trigger line). For analog lines, you can create the trigger using:

- The analog signal itself. This can be useful when debugging meter reads.
- A spot pulse generated by the Support Board

To create a trigger using only an analog signal:

1. Connect the analog line to AnalogLine1 using the instrument's ConnectToSyncPanel method.
2. Connect your external instrument to ANALOG SYNC.
3. Set your external instrument to trigger on the analog transition you expect.

For example, if your DCVI channel delivers 4V when gated on, set the external instrument to trigger on a transition of -4V.

To trigger on an event generated by the Support Board (spot pulse 1 or 2):

1. Connect the event to a digital Sync Panel line. For example:

```
TheHdw.SyncPanel.Connect tlFlexSyncSignalSpotPulse1, _
                         tlFlexSyncPanelDigitalLine1
```

2.	Connect your external instrument to DIG SYNC 1.
3.	Set your external instrument to trigger on a 1s pulse of +3.3V.
4.	Generate the event at the point of interest in your test program. For example:

TheHdw.SyncPanel.GenerateSyncPulse tlFlexSyncPulse1

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_sync.6.4

< >

Sync Panel : Programming the Sync Panel : Example

Example

The following example uses a DCTime channel to trigger an external instrument so that you can view the activity on a DCVI channel. Note that the DCTime and DCVI channels must come from different DC boards.

1.	Connect the DCVI channel to the analog Sync Panel line:

```
TheHdw.DCVI("Pwr1").ConnectToSyncPanel tlFlexSyncPanelAnalogLine1, _
tlDCVISyncSignalsVI
```

2.	Connect the DCTime trigger to a digital Sync Panel line:

```
TheHdw.DCTime("TimeOut1").ConnectToSyncPanel tlFlexSyncPanelDigitalLine1, _
tlDCTimeSyncSignalAsyncTrigger1
```

3.	Connect the trigger line of your external instrument to the DIG SYNC 1 connector on the test head.
4.	Connect the measurement line of your external instrument to the ANALOG SYNC connector on the test head.
5.	Set the external instrument to trigger on a 1s pulse of -3.3V.
6.	Run your test program and monitor the output on the external instrument.

When the DCTime asynchronous trigger occurs, your external instrument shows the DCVI channel's activity.

7.	Disconnect all channels from the Sync Panel. Use the Disconnect method. You must disconnect each Sync Panel line (trigger and data) in a separate statement:

```
TheHdw.SyncPanel.Disconnect(tlFlexSyncPanelAnalogLine1)
TheHdw.SyncPanel.Disconnect(tlFlexSyncPanelDigitalLine1)
```

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_sync.6.5

< >

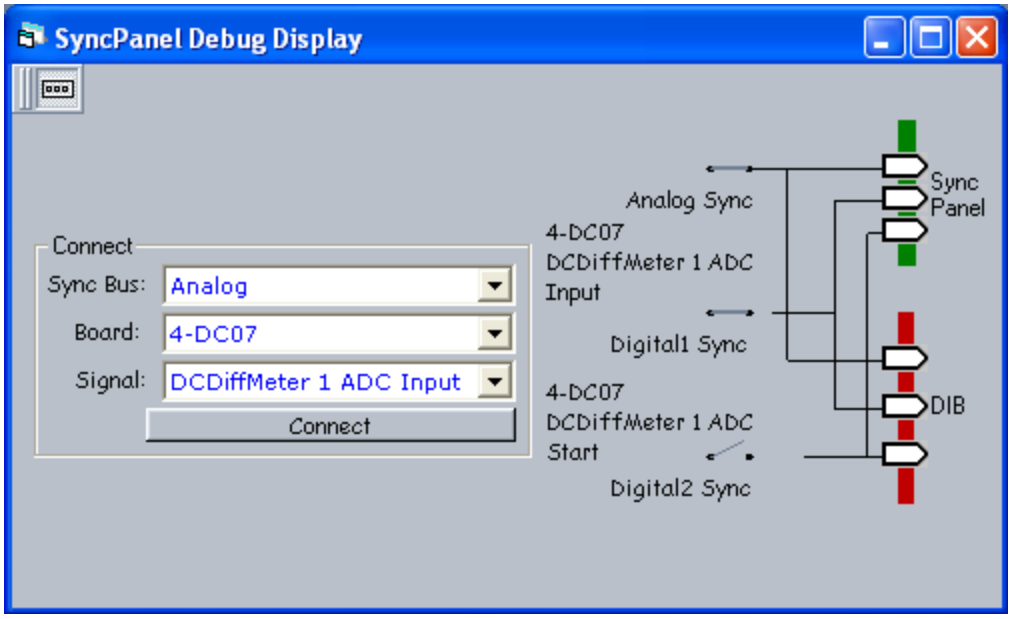
Sync Panel : Starting the Sync Panel Debug Display

Starting the Sync Panel Debug Display

The Sync Panel Debug Display lets you select instrument signals in an UltraFLEX test system and view them through the BNC connectors on the test head Sync Panel. The Debug Display provides the same functionality as the SyncPanel VBT syntax but with greater flexibility.

To launch the Sync Panel Debug Display, on the IG-XL Tools tab, in the Launch group, select Subsystems>SyncPanel.

You can also open the Sync Panel Debug Display from inside the TDE.



Sync Panel Debug Display

For more information, see:

- Using the Sync Panel Debug Display
- SyncPanel (VBT Syntax Reference)

SupportBoard_sync.6.6

< >

Sync Panel : Using the Sync Panel Debug Display

Using the Sync Panel Debug Display

You can view one analog and two digital signals simultaneously through the Sync Panel but only one signal at a time on each sync line.

To select a signal to display:

1. Choose a connection.

From the Sync Bus list, select Analog, Digital 1, or Digital 2.

These selections correspond to the BNC connectors on the test head labeled, respectively, ANALOG SYNC, DIG SYNC 1, and DIG SYNC 2. The same instruments and channels are available at both digital ports. Choose the connector to which you want to attach your oscilloscope or other external instrument.

2. Choose a board that contains the signal you want to display.

The Board list includes all instruments that can provide a Sync Panel Signal based on the Sync Bus you selected in the Sync Bus list. Each item shows the tester slot and the board type.

3. Choose an instrument signal.

The Signal list includes all signals from the selected board that can connect to the Sync Panel.

4. Click Connect to connect the signal to the Sync Panel or DIB.

5.	View the instrument signal with an oscilloscope or other external instrument either at the Sync Panel or at the DIB.

To view instrument signals on a DIB you have designed, you must place appropriate connectors on your DIB and connect them to the Pogo pin connections from the support board (see Typical Sync Panel Bus Connections).

To disconnect a port, click the relay that controls that port. For example, to disconnect the Digital 1 port, click the relay labeled Digital 1 Sync.

SupportBoard_theory.2.1

<	>

Support Board Theory of Operation

Support Board Theory of Operation

This section describes the hardware concepts and the theoretical practice of the Support Boards to help you understand their basic functionality. The description emphasizes theory and concepts that assist in programming.

The two UltraFLEX support boards are located in test head slots 24 and 25. Each board has an attached power board and a mechanical stiffener frame. The support board's main function is to support other hardware in the test system.

The following features are programmable:

- Utility data bits
- User power
- DIB and probe card EEPROMs

The following topics are available:

- Support Board Signals
- Primary and Secondary Support Boards

The following sections provide additional information about Support Board functions:

- Utility Data Bits Theory of Operation

- DIB Power Theory of Operation

- EEPROM Theory of Operation

For information on Support Board performance specifications and standards, see UltraFLEX System Specifications.

SupportBoard_theory.2.2

<	>

Support Board Theory of Operation : Support Board Signals

Support Board Signals

This section describes the characteristics of the signals found on the UltraFLEX support boards.

DIB EEPROM	The Primary support board provides chip selects to enable up to eight EEPROMs. The chip selects are labeled ID0 to ID3 on the Primary support board. You can use any +3.3V EEPROM.
Digital Ground	Support board ground.
Device Ground Sense	A sense signal that tracks the device ground on the DIB with respect to the support board ground. Tie to device ground.
CLK100	Freerunning 100MHz digital signal available to the user.
HV_CONNECT	On the Primary support board. Indicates to the tester whether the DIB is pulled down and seated properly before the DIB is powered up. Both HV_GATE and HV_CONNECT must be tied to the DIB ground or certain instruments (that is, User Power, DC V/Is) will not provide power to the test head.
HV_GATE	On the Secondary support board, indicates to the tester whether the DIB is pulled down and seated properly before the DIB is powered up. Both HV_GATE and HV_CONNECT must be grounded. Both HV_GATE and HV_CONNECT must be tied to the DIB ground or certain instruments (that is, User Power, DC V/Is) will not provide power to the test head.
ID PROM	See DIB EEPROM.

SupportBoard_theory.2.3

< >

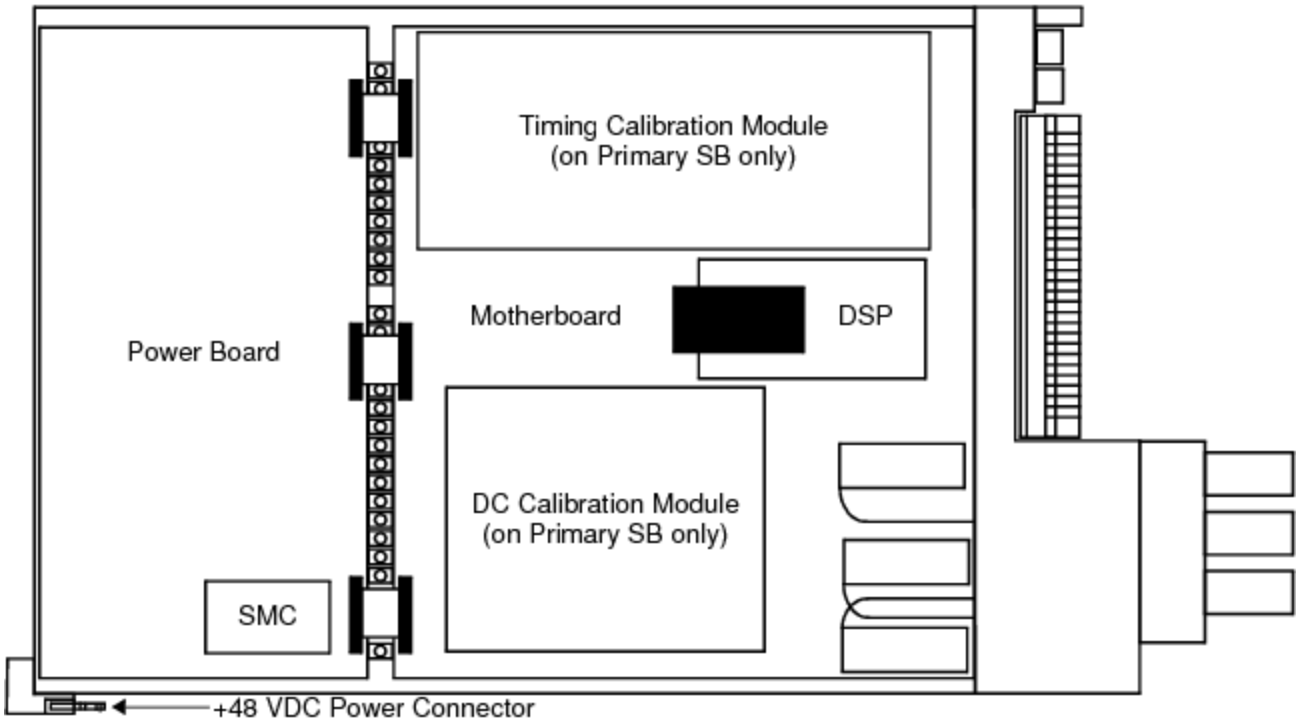
Support Board Theory of Operation : Primary and Secondary Support Boards

Primary and Secondary Support Boards

Two support boards exist in the UltraFLEX test head. One board is designated the Primary support board and the other is the Secondary. The support boards are located in slots 24 and 25 of the test head. This places them at or near the ends of the test head, in opposite hemispheres, which allows for optimal routing of signals to and from instrument slots. The signals are distributed on the support board backplane connector to ease routing. See Test Head Slot Numbering Convention.

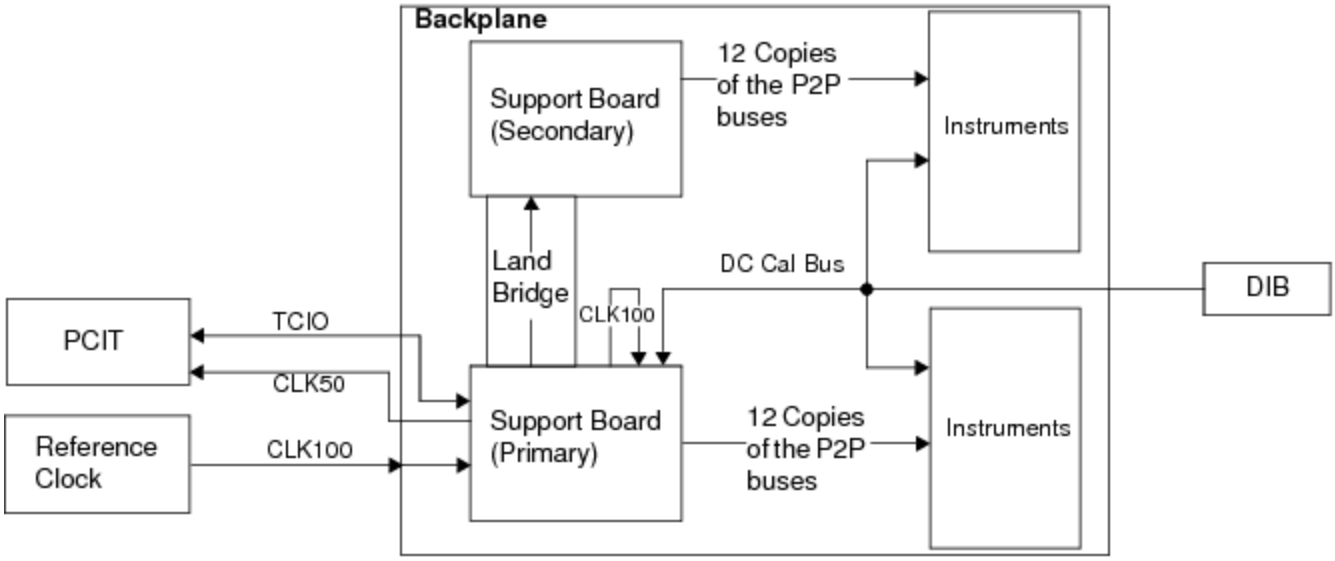
At the top of the board is a 100 wafer, eight-row VHDM connector that plugs into the UltraFLEX backplane. This connector receives signals from and distributes signals to instrument slots, the computer, the MCS, and the other support board.

A second 25 wafer section of eight-row VHDM connector interfaces to the backplane for signals that go to or from the DIB. This is the I/O connector. A cable assembly attaches to a back-to-back VHDM connector on the other side of the backplane and connects support board signals to two locations on the DIB. This cable assembly is installed in the test system during construction and is not configuration dependent. The cable assembly is split into two, so that some signals go to one DIB location and other signals to the other DIB locations.



Support Board Block Diagram

A “land bridge” in the center of the backplane allows critical signals to flow between the support boards on the shortest path. Less critical signals between the support boards are routed around the ends of the backplane.



Support Board Connections Block Diagram

Most of the signals between the support boards and instruments are distributed point to point, with each instrument getting its own buffered copy or the support board receiving a unique copy from each instrument. Most pins on the 800 pin backplane connector are dedicated to this distribution. The circuitry to support the point-to-point strategy does not take up much of the support board area due, in part, to the use of high pin count FPGAs that, in a single package, support the distribution to all 12 or 18 slots driven from each support board.

SupportBoard_utilbits.3.01

< >

Utility Data Bits

Utility Data Bits

Utility data bits (UDB) are sources that provide low speed logic signals for general use such as driving relay coils or slow speed read back of the logic state on DUT pins. Each UltraFLEX test system has 128 UDBs, and each bit can sink up to 150mA.

See Utility Data Bits in the DIB Design Guidelines for the locations of the UDBs.

This section includes the following topics:

- Utility Data Bits Theory of Operation
- Utility Data Bits Programming
- Utility Bits Debug Display
- Utility Data Bit Applications

< >

SupportBoard_utilbits.3.02

< >

Utility Data Bits : Utility Data Bits Theory of Operation

Utility Data Bits Theory of Operation

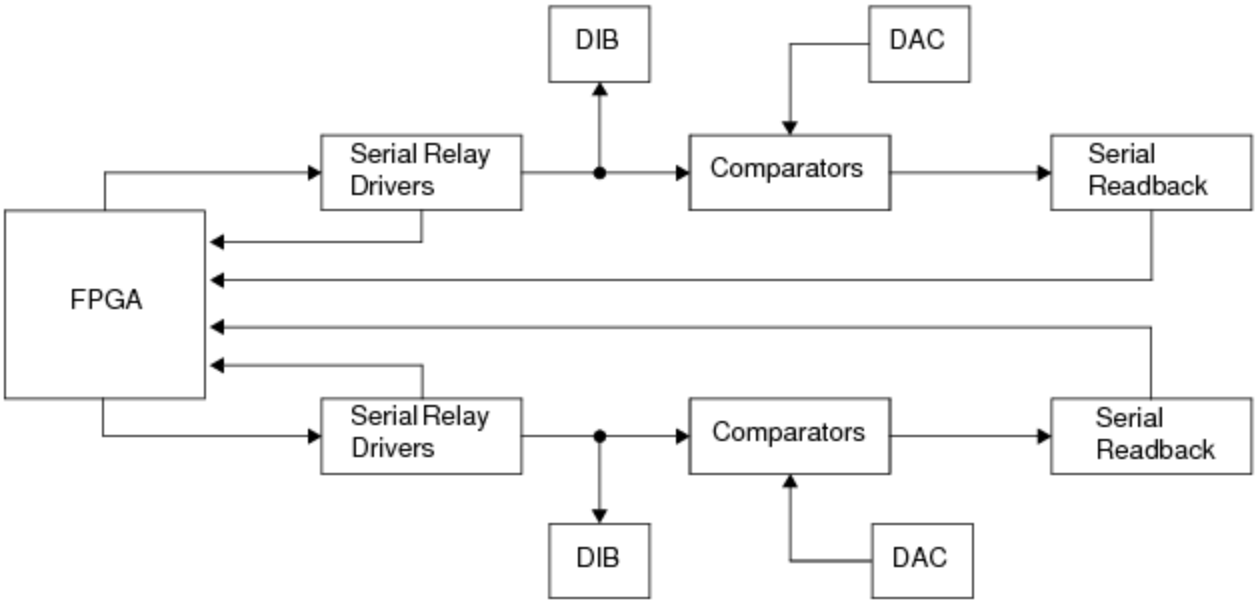
Utility data bit circuitry is included on each master support board. Each utility data bit circuit has a driver, comparator, and circuitry to change the state of the driver and read the state of the comparator. The drivers have 1Kohm pull-ups to 3.3V and diode clamps for voltage protection. The comparator threshold levels are not independent: the levels are programmable from about 0V to about 12V.

Each bit can sink up to 150mA. The current limit for eight bits is 1A. The table lists the approximate programmable range.

Range Parameters

Parameter	Value
Detect threshold Range	0 - 12V
Current sink capability	150mA
Current limit	1A
Output voltage range	0 to 12V
Inductive voltage clamp	-0.8V to 50V

The following figure shows a block diagram of the UltraFLEX utility bits.

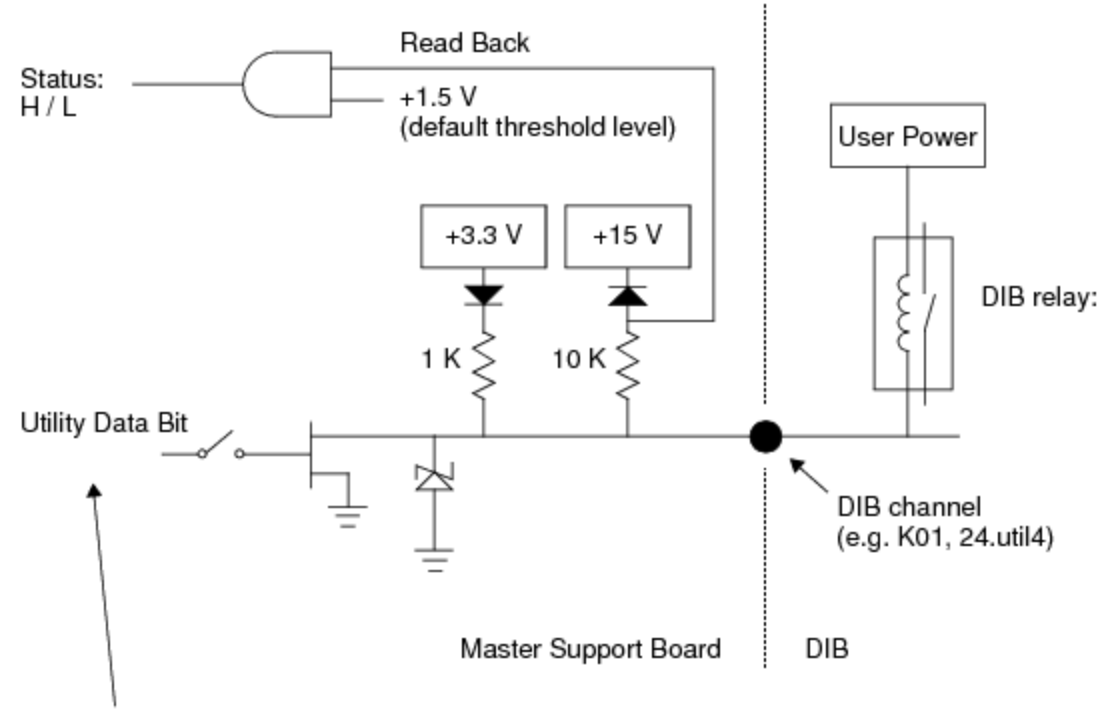


Utility Data Bits Block Diagram

UDBs are isolated from test system ground. Connect UDB ground to DIB ground and to the return of the User Power Supply used to power the relays.

UDB drivers can control the state of relays on the DIB. When a utility data bit is programmed to "on," the driver pulls the output low turning the relay on. When a utility data bit is programmed to "off," the driver pulls the output high turning the relay off.

The following example shows the connection of a UDB to a DIB relay.



Programming state: 1 (On) = Drive low, turns on the DIB relay
Programming state: 0 (Off) = Drive high, turns off the DIB relay

Utility Data Bit Connection to DIB Relay

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_utilbits.3.03

< >

Utility Data Bits : Utility Data Bits Programming

Utility Data Bits Programming

The steps to program Utility Data Bits are:

1.	Program the pin map (Programming the Pin Map).
2.	Program the channel map (Programming the Channel Map).
3.	Program the UDBs using VBT functions.
4.	Include the VBT functions in a test procedure.
5.	Include the test procedure in a test instance.
6.	Include test instance in Flow Table sheet.

For information on UDB VBT syntax, see Utility in the VBT Syntax Reference.

< >

SupportBoard_utilbits.3.04

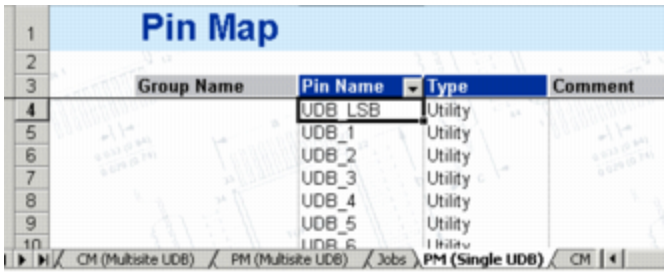
< >

Utility Data Bits : Utility Data Bits Programming : Programming the Pin Map

Programming the Pin Map

Use a pin map to assign a pin name to the Utility Data Bits.

Enter the Utility Data Bit pin names under the Pin Name column. The Type must be Utility.



Pin Map Sheet Showing Utility Data Bits

< >

SupportBoard_utilbits.3.05

< >

Utility Data Bits : Utility Data Bits Programming : Programming the Channel Map

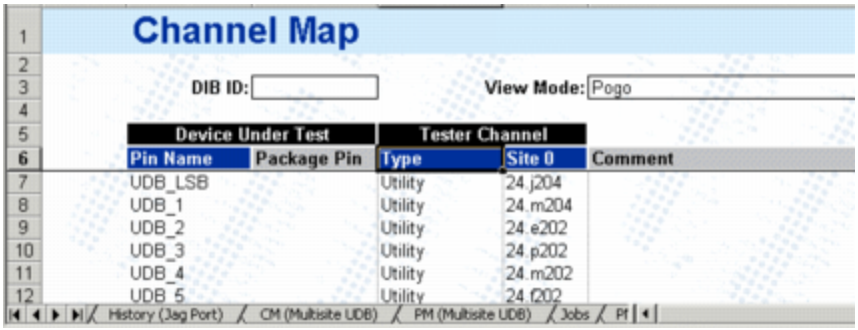
Programming the Channel Map

Use a channel map to assign Utility Data Bits to the pin names in the pin map sheet.

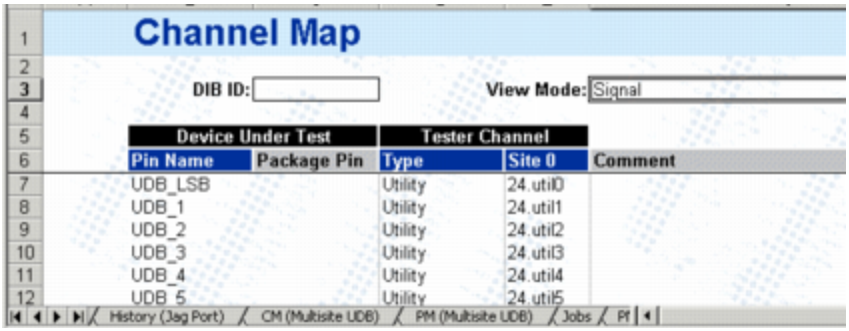
Enter Utility Data Bit pin names in the Pin Name column and the DIB pad to which it is connected in the Site column. The Type must be Utility.

The channel map can display the Utility Data Bits in either of two modes:

Pogo	In the Pogo view, you must enter the Pogo connections.
Signal	In the Signal view, for each pin you can enter the name of the signal, such as 24.util0, 24.util1, and so on, where 24 is the slot number of the support board with the Utility Data Bit to be used. In the Signal view, you need not know the Pogo connections.



Channel Map Using Pogo View



Channel Map Using Signal View

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_utilbits.3.06

< >

Utility Data Bits : Utility Data Bits Programming : Utility Bits Reset Values

Utility Bits Reset Values

When the utility bits are reset all utility bits are cleared and the threshold is set to 1.5V for each.

< >

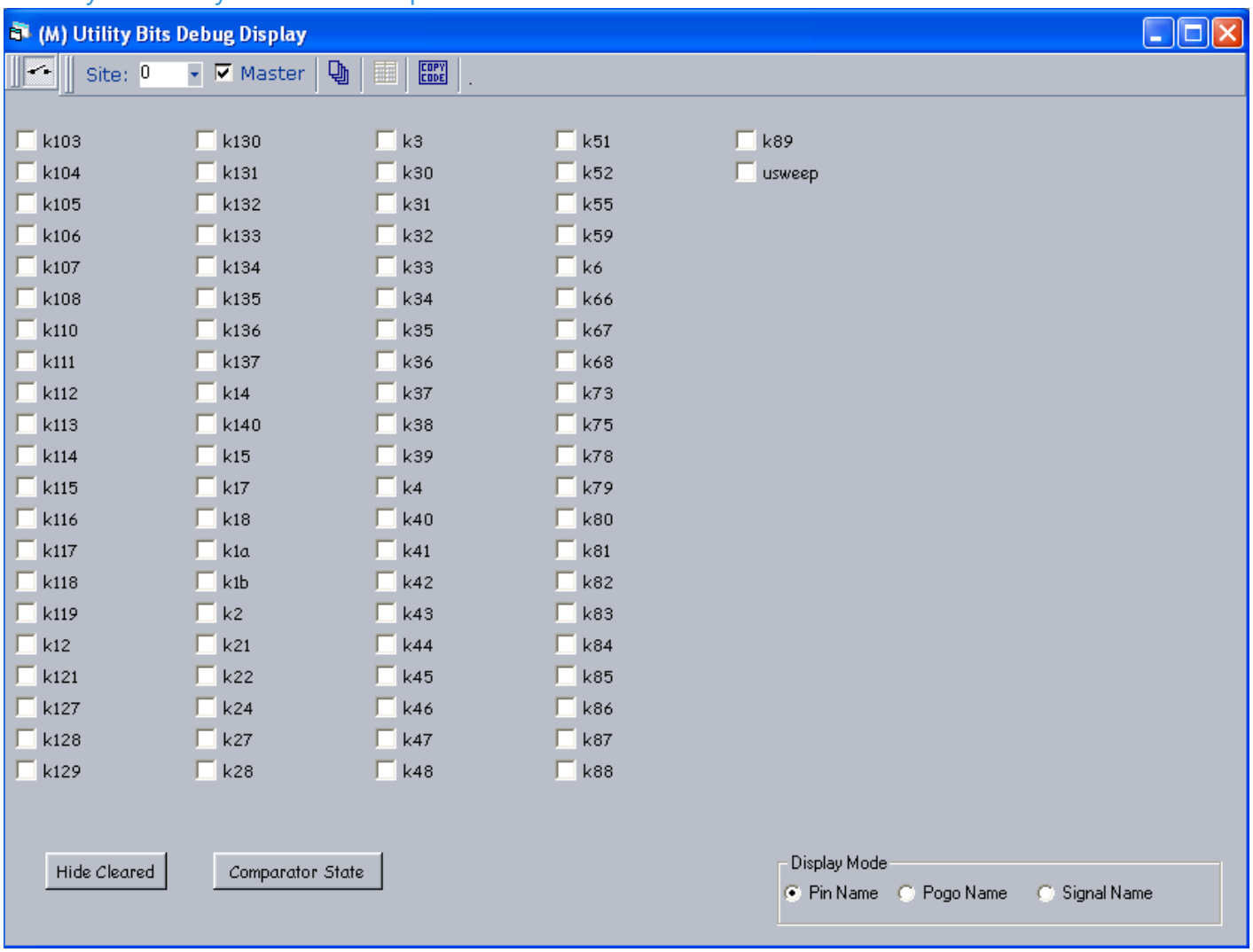
SupportBoard_utilbits.3.07

< >

Utility Data Bits : Utility Bits Debug Display

Utility Bits Debug Display

The Utility Bits Debug display is an interactive GUI that allows you to see the current state of the Utility Data Bit, turn it on or off, change the comparator threshold value, or view the values read back by the Utility Data Bits comparator.



Utility Bits Debug Display

For more information on using debug displays, see the following:

- Using Debug Displays in TDE
- Producing Code from Debug Displays

Bringing Up the Display

To display the values for utility data bits, a test program must have utility data bits declared, it must validate without errors, and then you must select a site to be debugged from the Site menu. All changes appear in real time.

To launch the Utility Bits Debug Display, on the IG-XL Tools tab, in the Launch group, select Debug Displays>Utility Bits.

You can also start the display from with in the TDE.

Display Parts

The display consists of the main window that displays each of the programmed pins with a check box. Several buttons, menus, and a status bar also appear in the display.

The debug display shows all programmed data bits organized in columns and listed by pin name in alphabetical order. Each has a check box that displays the utility data bit status:

- Checked: 1 or “on”
- Unchecked: 0 or “off”

To turn a Utility Data Bit on, move the pointer over its box and click it. A check mark appears in the box to indicate the Utility Data Bit is on. You can also tab through each item and select using the Enter key.

You can resize the main window to display as many programmed utility data bits as possible. If not all pins fit on the screen, a horizontal scrollbar is present.

When you select a utility pin on the display, the second panel of the status bar displays the channel information for the pin you select.

Controls

The display includes the following controls:

Hide Cleared	Displays only the utility data bits that are checked in the main window as well as hiding the Comparator State button. When clicked, this button changes to Show All.

Comparator State	Shows a read only display that displays the state of the comparator for each utility data bit: • H: Utility data bit compared state high • L: Utility data bit compared state low When clicked, this button changes to Show Programmed.
Comparator Threshold Voltage	When you click Comparator State, this text box appears. It displays comparator threshold voltage. To change the threshold voltage, click in the box, type the new value, and press the Enter key. You must press Enter for changes to take effect. Placing the mouse pointer over the comparator threshold box displays the allowable comparator threshold voltage values. While in the comparator state, the button Apply All Sites is disabled.
Display Mode	Selects the display mode for each utility data bit: • Pin Name: Name of the data bit in the test program • Pogo Name: Pogo pin used by the data bit • Signal Name: Signal name of the data bit

SupportBoard_utilbits.3.08

< >

Utility Data Bits : Utility Data Bit Applications

Utility Data Bit Applications

This section contains a sample application for programming the Utility Data Bits.

The example in this section does the following:

1.	Clear the Utility Data Bits (UDB).
2.	Turn on relays K4 and K6 on the DIB.
3.	Turn off relay K10 on the DIB.
4.	Read the state of UDB called K10.
5.	Change the threshold from 1.5V to 3.0V.
6.	Read the threshold value.

< >

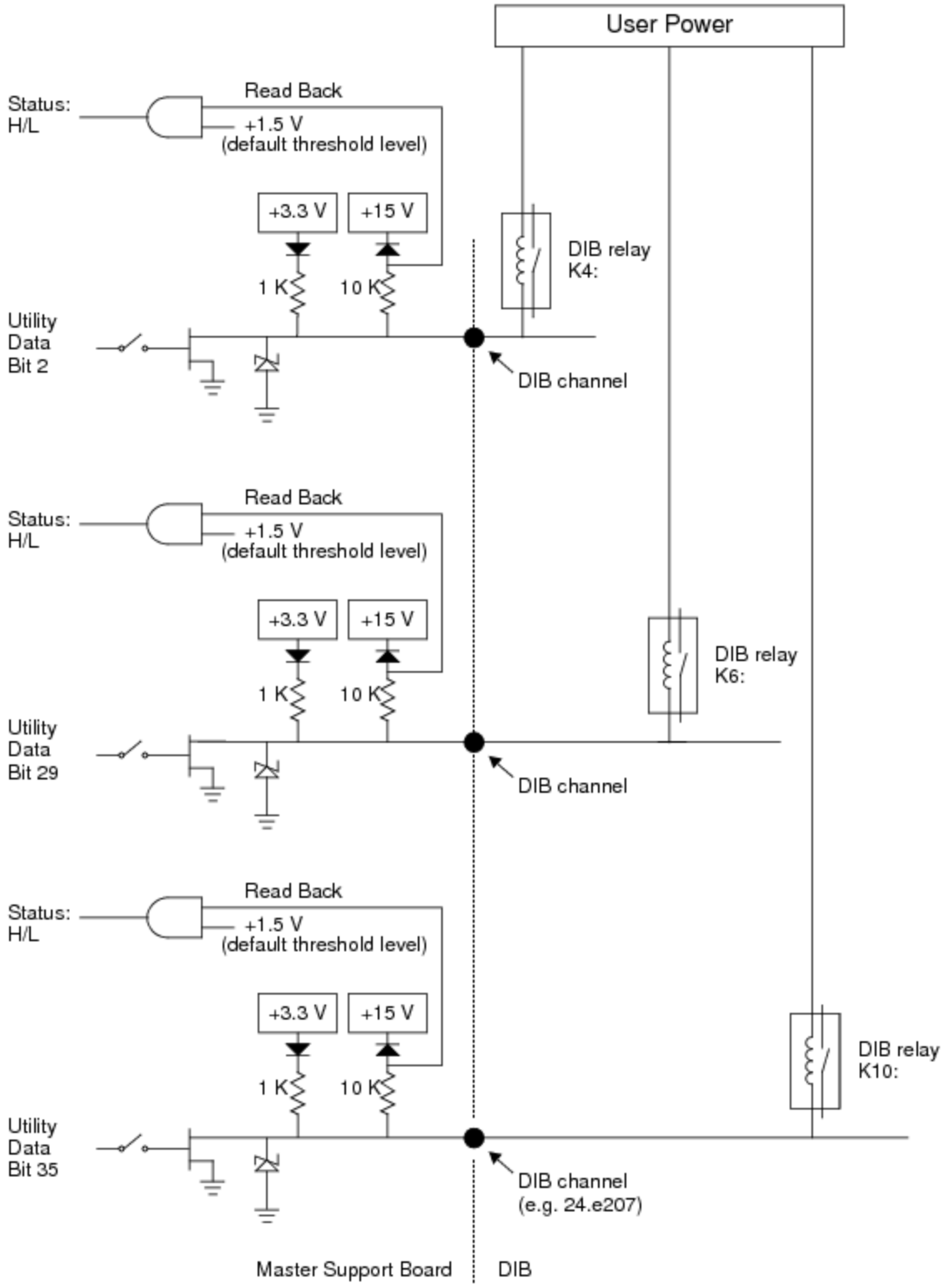
SupportBoard_utilbits.3.09

< >

Utility Data Bits : Utility Data Bit Applications : Set Up the Circuit

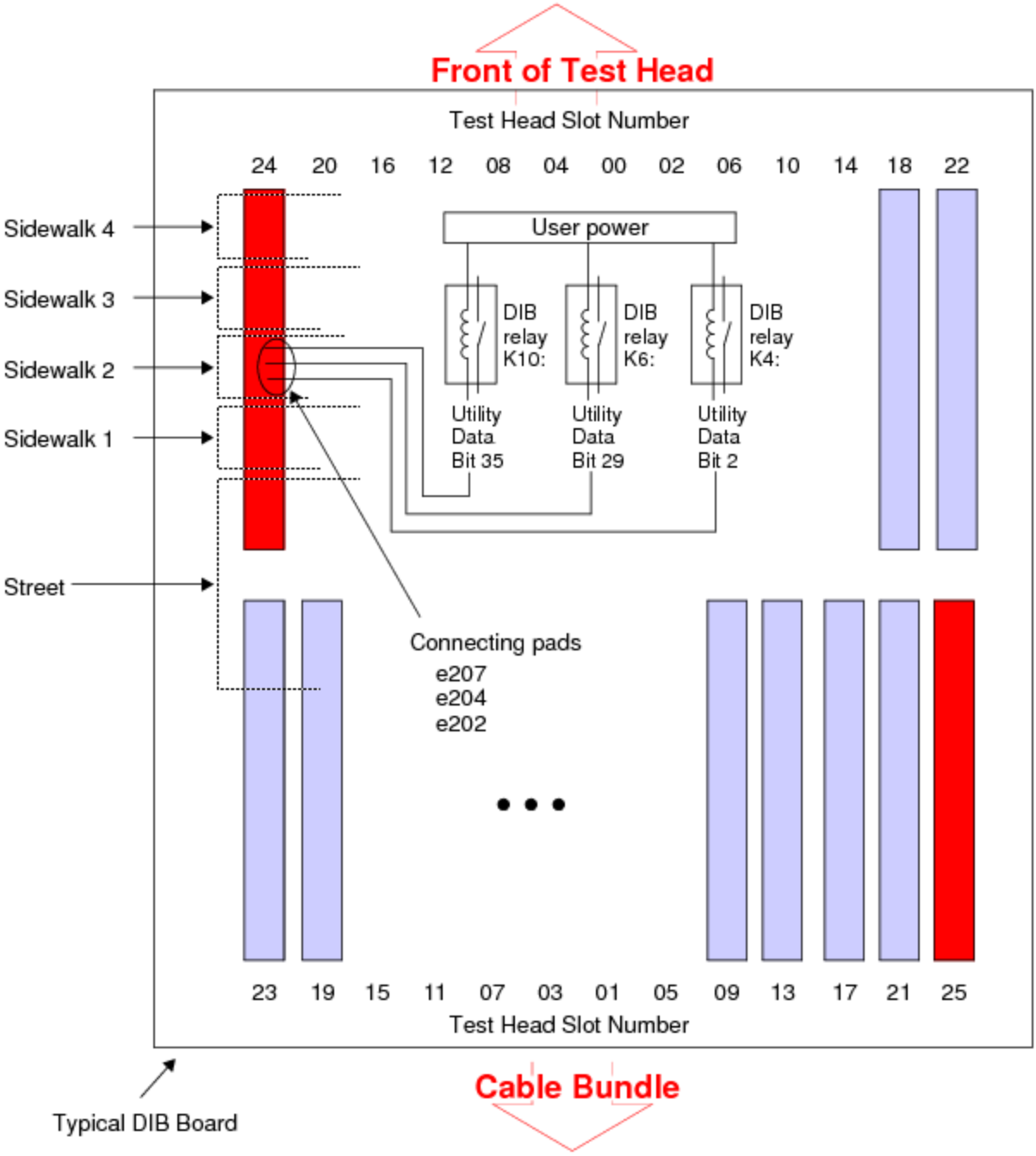
Set Up the Circuit

This figure shows the circuit for this example:



Block Diagram of Programming Example

This figure shows DUT connections and DIB wiring required for this test. See Utility Data Bits in the DIB Design Guide for the locations of the bits.



Typical DIB Layout for Programming Example

< >

2015 Teradyne, Inc. All rights reserved.

SupportBoard_utilbits.3.10

< >

Utility Data Bits : Utility Data Bit Applications : Set Up DataTool

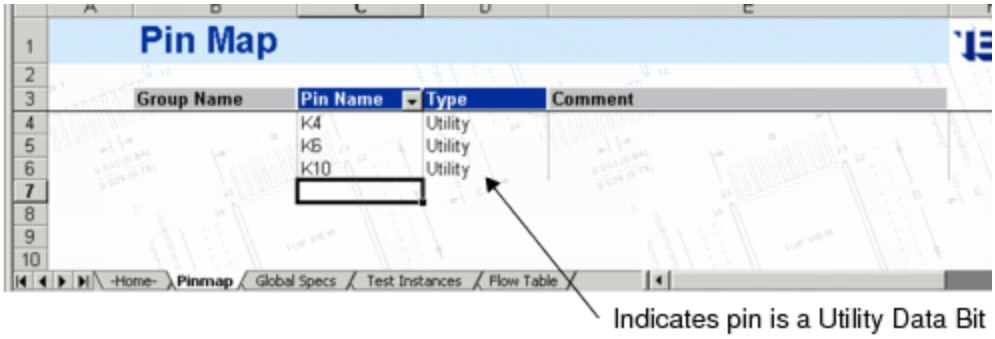
Set Up DataTool

The following DataTool sheets and data are used in this test:

- Pin Map sheet
- Channel Map sheet

Pin Map Sheet

Pins K4, K6, and K10 are described as Utility Data Bits.



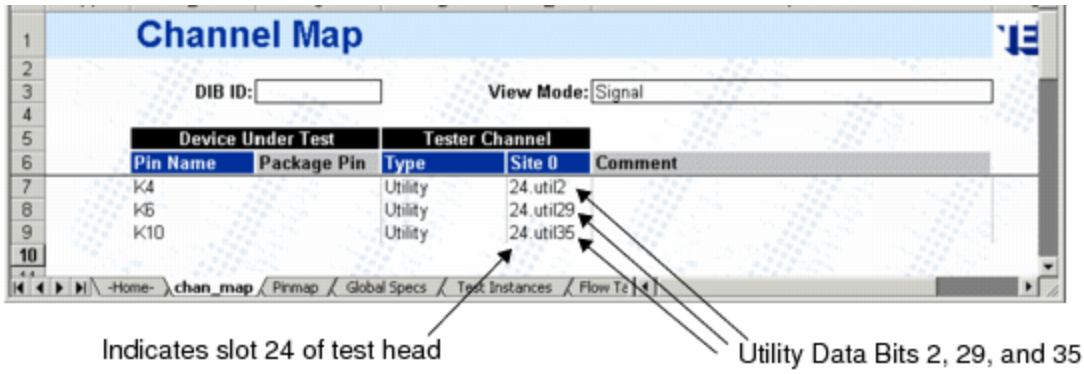
Pin Map Sheet

Channel Map Sheet

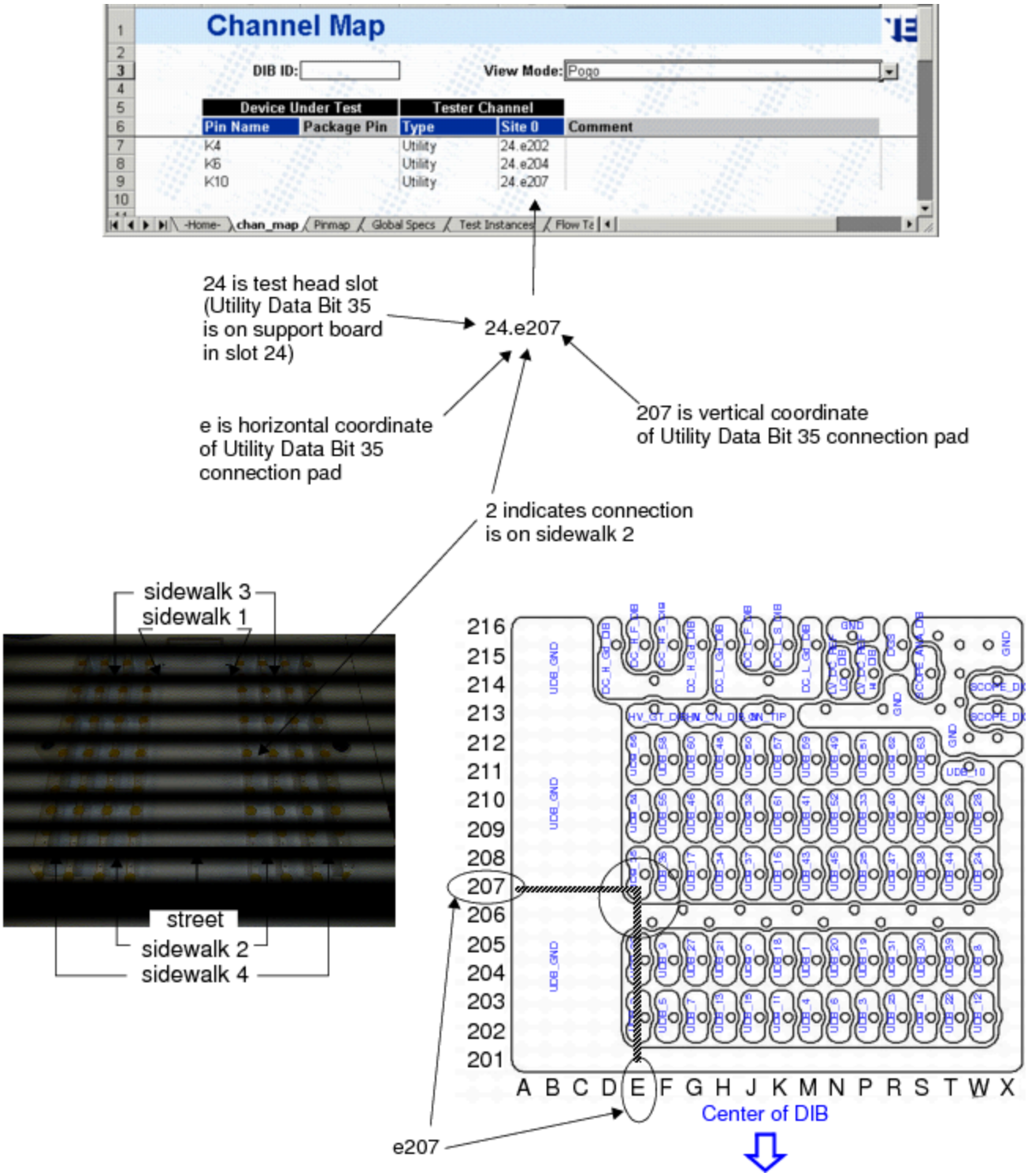
Utility Data Bit 2 is assigned to pin K4. UDB2 is on the support board in slot 24.

Utility Data Bit 29 is assigned to pin K6. UDB29 is on the support board in slot 24.

Utility Data Bit 35 is assigned to pin K10. UDB35 is on the support board in slot 24.



Channel Map Sheet, Signal View



Channel Map Sheet, Pogo View

VBT Code

The following VBT code performs this test. Cut and paste it into the VBT editor.

```
Public Function RelayControl() As Long
Dim UtilState as PinListData
Dim ThresValue as Double
' Clear all the utility bits to off and threshold set to 1.5 V
TheHdw.Utility.Reset
' Turn on K4 and K6 DIB relays
TheHdw.Utility.Pins ("K4, K6").State = tlUtilBitOn
' Turn off K10 DIB relays
TheHdw.Utility.Pins ("K10").State = tlUtilBitOff
' Check the state of a utility bit K10
' Returns a 0 for OFF state and 1 for ON state
UtilState = TheHdw.Utility.Pins ("K10").States (tlUBStateProgrammed)
' Change the value of threshold from 1.5 V to another value, max 12.0 V
' This threshold is used for debugging to view the comparator state
' on the debug display
TheHdw.Utility.Threshold = 3.0
' Read back the current threshold value
ThresValue = TheHdw.Utility.Threshold
End Function
```

< >

2015 Teradyne, Inc. All rights reserved.